# HOMEWORK ASSIGNMENT 3

CSCI 571 – Spring 2025


## Abstract

Angular, TypeScript, Bootstrap, Responsive Design, JavaScript in Server Side, Node.js, Express, AJAX, JSON, and Artsy API





Marco Papa

papa@usc.edu

# Assignment 3: Angular, TypeScript, Bootstrap, Responsive Design, JavaScript in Server Side, Node.js, Express, AJAX, JSON, and Artsy API

## 1 Objectives

1. Get experience with creating backend applications using JavaScript/Node.js on the server side with  framework.
2. Get experience with using Angular, TypeScript, and Bootstrap on the client side and creating responsive frontend.
3. Get experience with using HttpClientModule of Angular for AJAX.
4. Get experience with Artsy API.
5. Get experience with Google Cloud Platform (GCP).

## 2 Assignment Description Resources

1. Assignment Description Document (This document)
2. Rubric (aka, Grading Guidelines)
3. Web Reference Video: https://youtu.be/fj7cPxLDiM8
4. Mobile Reference Video: https://www.youtube.com/shorts/eb504OLXJ7I
5. Piazza

## 3 General Directions

1. The backend of this Assignment must be implemented in JavaScript using Node.js Express framework (or Fastify/hono.js/ElysiaJS – however limited TA/CP support will be available). Refer to Node.js website for installing Node.js and learning how to use it. Have a look at "Getting started" guides in Express website to learn how to create backend applications using Express. Axios and other libraries can be useful to make requests from your Node.js backend to Artsy servers. **Implementing the backend in anything other than Node.js will result in a 4-point deduction.**
2. The frontend of this Assignment must be implemented using the **Angular** framework. Refer to Angular setup docs for installing Angular and creating Angular projects. Angular "Tour of Heroes" app tutorial is a very good tutorial to see different Angular concepts in action. **Implementing the frontend in anything other than Angular will result in a 4-point deduction.**
3. You are expected to create a responsive website. For that reason, we require you to use **Bootstrap**, a CSS framework for responsive, mobile-first web development. It will save you from the burden of dealing with CSS peculiarities (Some of you may have felt like this in Assignment 2) and the website you create will be responsive automatically if you develop within the framework provided by Bootstrap. Please refer to Bootstrap docs for reference (especially look

at “Layout” section, and the components that you want to use). Refer to this post to learn how to add Bootstrap to Angular projects. **Not using Bootstrap will result in a 4-point deduction.**

4. The backend of this Assignment must be deployed on Google Cloud. The backend should serve the frontend as well as other endpoints you may define. Please refer to the instructions on D2L Brightspace for deploying Node.js applications to any of the cloud services.
5. You must refer to the Assignment description document (this document), rubric, the reference videos and instructions in Piazza while developing this Assignment. All discussions and clarifications in Piazza related to this Assignment are part of the Assignment description and grading guidelines. Therefore, please review all Piazza threads before submitting the assignment. If there is a conflict between Piazza and this description and/or the grading guidelines, answers and clarifications in Piazza supersede the other documents.
6. The Assignment will be graded using the latest version of the Google Chrome browser. Developing the Assignment using the latest version of Google Chrome is advised.

# 4 Description

In this Assignment, you are going to create a web page that allows its users to search for artists using the Artsy API. The results of the search will be listed as a list of artist cards. When a user clicks on an artist card, details about the chosen artist will be shown underneath. The details will contain two tabs, one containing information about the artist (name, birthday, deathday, nationality and biography) and other containing artist’s artworks. Each artwork belongs to several categories which will be shown when the categories button below the artwork is clicked.

## 4-1 Artsy Developer Registration (Same as Assignment 2).

To use Artsy API, you need to create a developer account in Artsy, create an app and get a client ID and a client Secret.

Download the PDF file named **Artsy REST API Setup** from D2L Brightspace to create an account, obtain the credentials to use the Artsy APIs, and test the APIs using Postman.

## 4-2 Artsy API Endpoints

The full list and documentation of Artsy API endpoints is given in https://developers.artsy.net/v2/. In this Assignment, you will use 5 Artsy API endpoints: Authentication, Search, Artists, Artworks, and Genes. The Artworks and Genes API endpoints are new. Also, some endpoints will be used for multiple purposes – with different sets of parameters. The Authentication, Search and Artist endpoints are the same as the ones used in Assignment 2.

Search, Artists, Artworks, and Genes endpoints require the **X-XAPP-Token** header, which you will get through the Authentication endpoint.

### 4-2-1 Authentication Endpoint (Same as Assignment 2)

You can find the details about this endpoint in Assignment 2.

### 4-2-2 Search Endpoint (Same as Assignment 2)

You can find the details about this endpoint in Assignment 2. As per Assignment 2, only entities of type “artist” should be returned.

### 4-2-3 Artists Endpoint (Same as Assignment 2)

You can find the details about this endpoint in Assignment 2.

You will use an additional variant (/artists) of this endpoint for similar artists.

Example request: https://api.artsy.net/api/artists?similar_to_artist_id=4d8b928b4eb68a1b2c0001f2

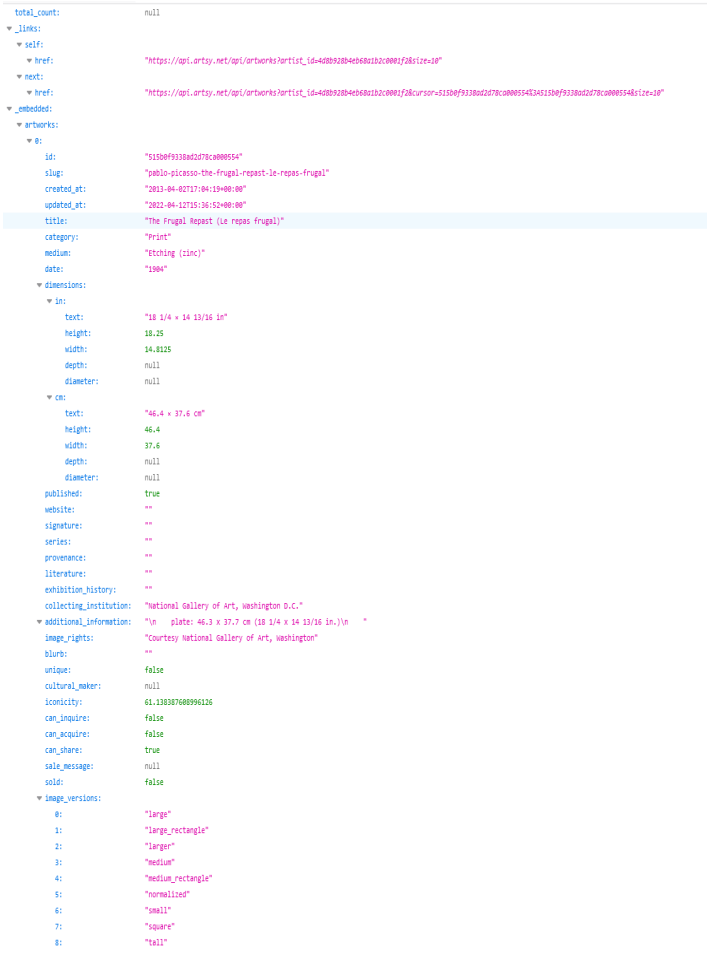
### 4-2-4 Artworks Endpoint

The **Artworks endpoint** is used to get a list of artworks that belong to other entities (i.e., artists, partners, shows, collections, users). In this Assignment, we are interested in retrieving the artworks of a specific artist. The Artworks endpoint has the base URL https://api.artsy.net/api/artworks. You send your request using the HTTP GET method. The request accepts a query parameter **artist_id** denoting the ID of the artist, artworks of which are being retrieved. Like the search endpoint, the maximum number of results can be set by the **size** parameter, which has the maximum value of 10. In this Assignment, you will always set it to 10. Like the search and artists endpoints, you should set X-XAPP-Token header in your request to the token returned from the Authentication endpoint.

**Note**: this search request may take several seconds to return results.

Example request: https://api.artsy.net/api/artworks?artist_id=4d8b928b4eb68a1b2c0001f2&size=10

The server returns a JSON result like the sample given below.



The artworks are listed in [“_embedded”][“artworks”] field. We are interested in [“id”] (ID of the artwork), [“title”] (name of the artwork), [“date”] (creation year of the artwork) and [“links”][“thumbnail”][“href”] (image of the artwork) fields of each artwork result.

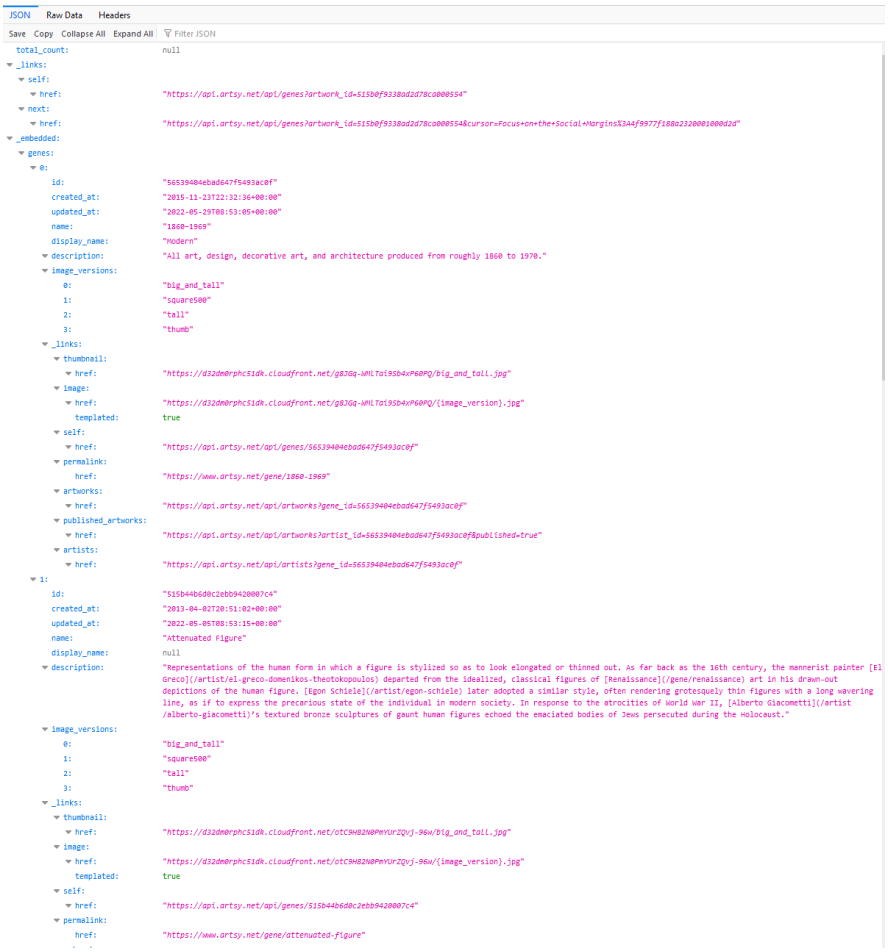
Similar to search results, artworks results are paginated, meaning that the endpoint returns a specific number of results after each request (set by the size parameter) and if the remaining results are wanted, another request can be made to the link in the [“_links”][“next”][“href”] field of the response. In this Assignment, you will only use results from the first page and will not make additional requests.

### 4-2-5 Genes (Categories) Endpoint

The **Genes endpoint** is used to get metadata about artists or artworks in Artsy. You will use it to get metadata about artworks. Since gene is a strange word for this context, we use the word “category” instead. The Genes endpoint has the base URL https://api.artsy.net/api/genes. You send your request using HTTP GET method. The request accepts a query parameter **artwork_id** denoting the ID of the artwork for which categories are being retrieved. Like the search, artists and artworks endpoints, you should set the X-XAPP-Token header in your request to the token returned from Authentication endpoint.

Example request: https://api.artsy.net/api/genes?artwork_id=515b0f9338ad2d78ca000554

The server returns a JSON result like the example given below.



The categories are listed in ["_embedded"]["genes"] field. We are interested in ["name"] (name of the category) and ["_links"]["thumbnail"]["href"] (image of the category) fields of this response.

Similar to search and artworks results, gene results are paginated, meaning that the endpoint returns a specific number of results after each request (set by the size parameter) and if the remaining results are wanted, another request can be made to the link in the ["_links"]["next"]["href"] field of the response. In this Assignment, you will only use results from the first page and will not make additional requests.

### 4-3 Gravatar Service Overview

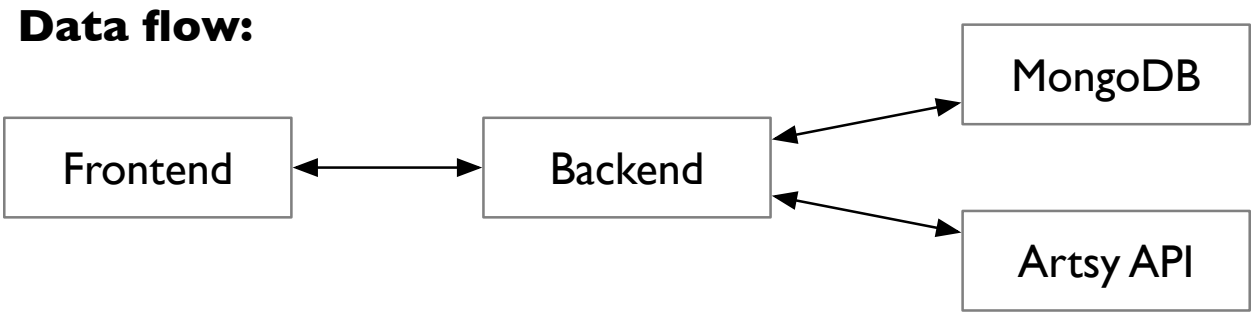
In this assignment, we will work with users and display their profile. Avatar is part of it. Some services have a functionality to upload profile pictures, however this could be very cumbersome. Instead, we will use a public profile pictures service!

**Gravatar** is an open service designed to store user public avatars and user public profiles. It is integrated into many services, including Wordpress. You will show the gravatar's profile image of a user in this assignment. There is no need to call any APIs from the backend. Instead gravatar matches the avatar by hash of the email, so you will need to calculate a **SHA1 hash** and form the correct image URL. You will need to generate and store this URL on the backend and return it to the frontend as part of the user's profile (see Authentication section).

If there is no avatar image matched, the service will generate a distinguishable visual pattern. More details and examples can be found at https://docs.gravatar.com/api/avatars/images/ .

### 4.4 System Overview (Similar to Assignment 2)

The system contains three components: 1) browser (frontend), 2) **Node.js/Express** application (backend) and 3) Artsy servers. You will implement the frontend and the backend. Your backend will have two pieces of functionality: serving the frontend static files to the browser and responding to the frontend's AJAX requests by fetching data from the Artsy servers. You will not directly call Artsy API from the frontend JavaScript, as it requires disclosing Client Id and Client Secret (and/or XAPP_TOKEN) to the public. The data flow diagram after an AJAX call is shown below.




**Sending a front-end request (that returns data) to any service except your backend will result in a 4-point penalty. Please do not directly call the Artsy API endpoints from your frontend.**

All requests from frontend to backend must be implemented using the HTTP GET method, as you will not be able to send us sample backend endpoint links as a part of your submission if you use HTTP POST.

## 4-5 Database

In this assignment we will store users and their preferences in a database. For these purposes, we will use a NoSQL database, MongoDB. MongoDB is also available as a Cloud Service, called **MongoDB Atlas**.

*MongoDB Atl*as is a source-available, cross-platform, document-oriented, DBMS. It is classified as a NoSQL DBMS (NoSQL Database Management System). A NoSQL DBMS is a non-relational database that is designed to handle large-scale, distributed, and flexible data storage. Unlike traditional SQL (relational) databases, NoSQL databases do not require fixed schemas, tables, or structured relationships.

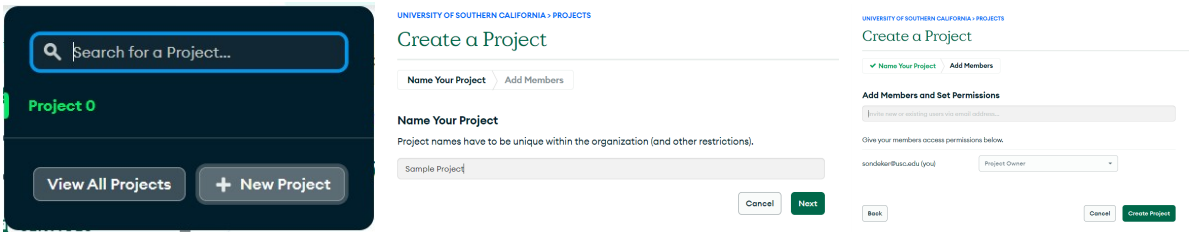
MongoDB Atlas uses JSON-like documents with optional schemas. For more information, see:

https://www.mongodb.com/docs/

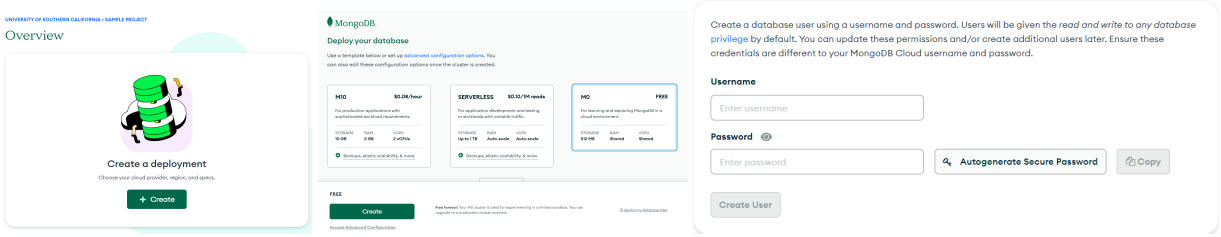
Also, see MongoDB on Google Cloud: https://www.mongodb.com/mongodb-on-google-cloud

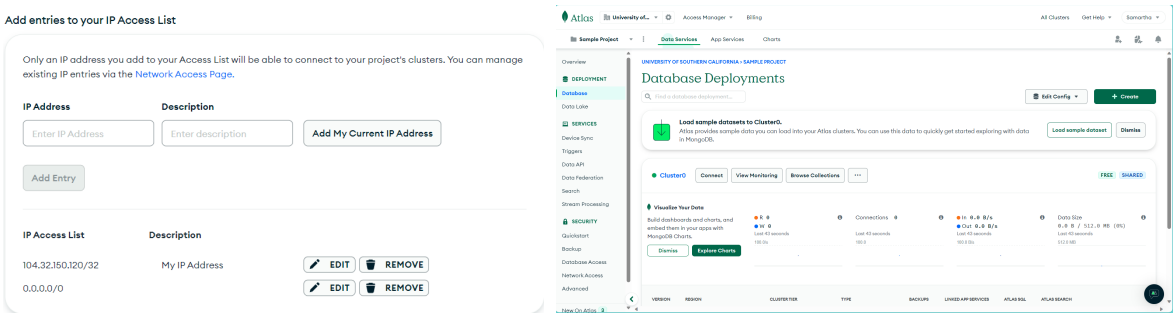
Once you set up an account in MongoDB Atlas, you will have to create a project to store your databases. Below are the steps to set up a project and create a database.

Follow these steps to create a project in which you can store databases for your application.

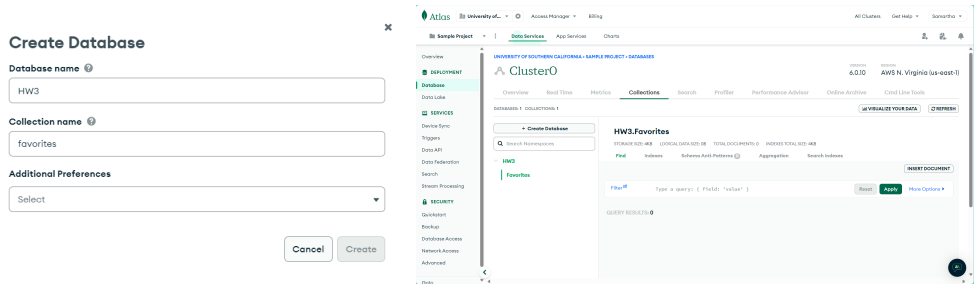



Follow these steps to create a deployment for the project by **creating a cluster** and providing user access and network access for the same. This would allow your application, running on a different server, to connect to MongoDB Atlas in the cloud.

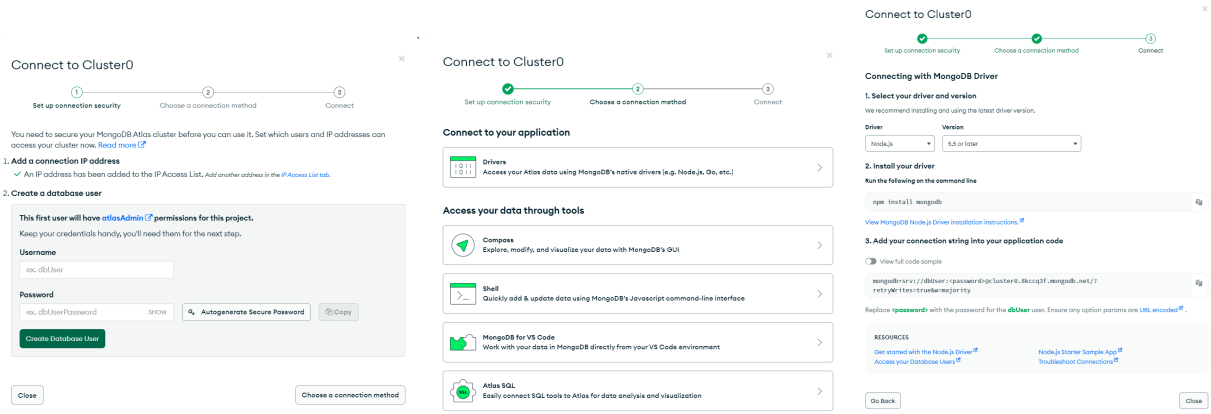





Follow these steps to **create a database** and a **collection** in that database. MongoDB stores data records as documents (specifically in BSON format) which are gathered in collections. A database stores one or more collections of documents.




Follow these steps to provide the instructions on how to **connect to your MongoDB database** from your application.




Once you have set up the database and the collection, you can connect to the database by adding the MongoDB node driver to your application. For more information on how to add the driver and run queries on the database, refer to [MongoDB Node Driver — Node.js.]

Make sure to set up accesses in the “Database Access” and “Network Access” side menu to give proper access to your client and allow certain IP addresses to access your database and collection(s). We recommend creating a dedicated user that will only have “ReadWrite” access to your collection(s) and set up an IP filtering such that it would match your IP for local development and the IP of cloud deployment. WE DO NOT RECOMMEND HAVING 0.0.0.0/0 entry – this will allow anyone to try and access your MongoDB instance.

## 4-6 Authentication

In this assignment we will support two types of clients: Guests and Authenticated users. Part of the functionality will be available (and visible) only to users that are logged in. In short, authenticated users can (in addition to guest users' privileges):

1) See "similar artists" section
2) Have a favorite list of artists (separate favorites page and add/remove buttons)

You will need to implement 4 actions (endpoints):

1) Registration (available only to guests)
2) Login (available only to guests)
3) Log out (available only to authenticated users)
4) Delete account (available only to authenticated users)

Moreover, your frontend should track (know) if the user is logged in and show information about current user (if any).

Auth functional requirements:

1) Auth state should be persistent across page reloads (F5/Cmd+R)
2) Auth state should be shared across tabs
3) Information belonging to user (Profile, favorites) should match the current user
4) Log out functionality should permanently change the auth state (ex: if you log out and reload the page, you should be still logged out, regardless of the tab)
5) Visible elements should respect the state of authentication (elements that are only available to logged users should be hidden on log out and vice-versa)

In this assignment we will use JWT tokens and Cookies.

### 4-6-1 Registration Procedure

The frontend should send a request to the backend containing all the fields necessary to register a user. The backend should validate the data. If there is another registered user with the same email or some fields are missing or contain incorrect values (though most of these errors should be caught on the frontend, see below discussion), a proper error message should be returned. The frontend will rely on this message to display all validation errors for the corresponding fields.

Also, the backend should generate a profile image URL. Please refer to the Gravatar Section and Gravatar's documentation. https://docs.gravatar.com/api/avatars/hash/ . Be advised, that you can use Node.js's built-in "node:crypto" module to create a sha256 hash.

The backend should insert the user's data into the database. Users' data should minimally contain the following fields (you may change their names): fullname, email, password (hashed), profileImageUrl. You may have additional fields if necessary.

Storing passwords in plaintext in the database is not secure! To prevent passwords from leaking they are stored in an irreversible hashed form. One of the popular secure hash algorithms is bcrypt. One of the recommended implementations is the "bcrypt" npm package. Please refer to its documentation:

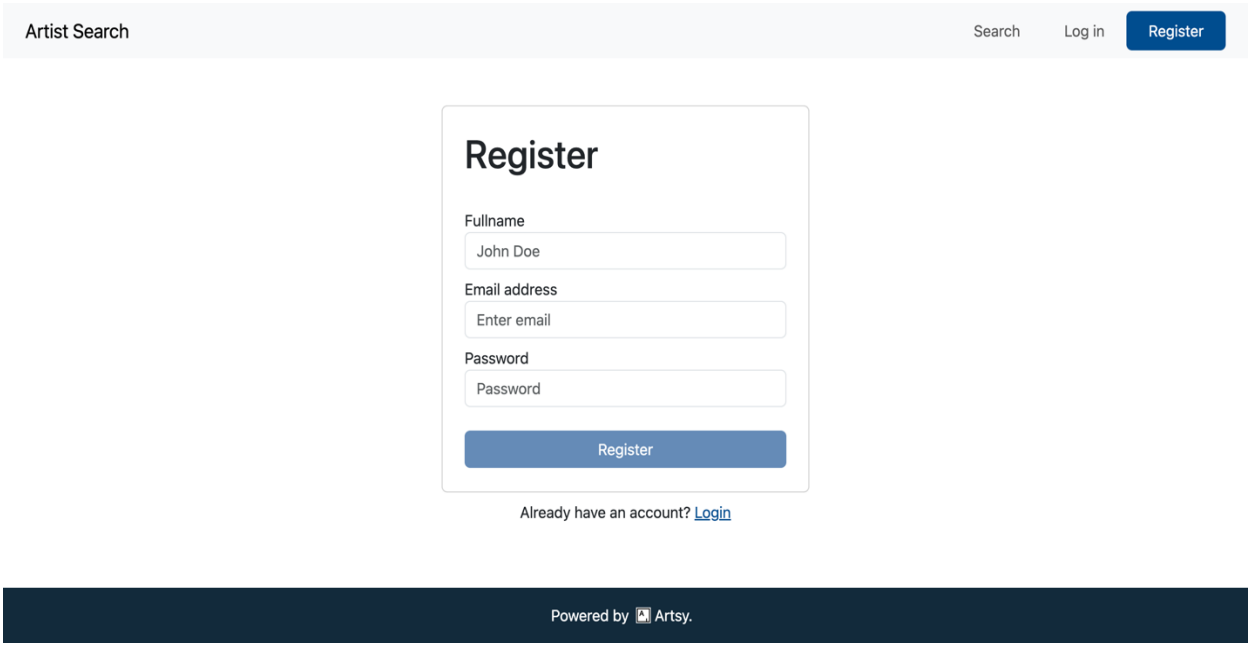
https://github.com/kelektiv/node.bcrypt.js?tab=readme-ov-file#esm-import

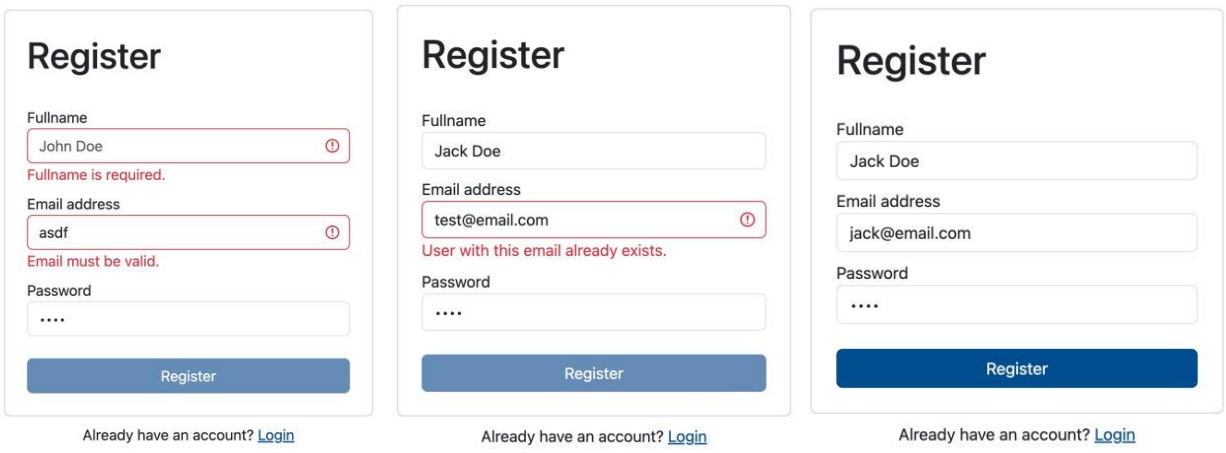
After the user's information is inserted in the database, we need to create and return the JWT token to the frontend. In this assignment we will use cookies to set, store and send these tokens. Token and cookies should be set to be valid for 1 hour.

The **registration page** (and the menu) should be available to unauthenticated users only and should match the screenshot below.

The form should have validation – all fields are required (use the REQUIRED attribute), the email field should contain a valid email address. This validation should be performed on the client (use the PATTERN attribute). Additionally, validation errors coming from the backend should also be shown under a corresponding field (please refer to the screen shot). Validation errors disappear once the field value has been modified (and does not contain a frontend-checked error anymore). The "Register" action button should be disabled while the form is empty or contains validation errors.

The login link should lead to the **login page**, as shown in the next page.






### 4-6-2 Login Procedure

The front end should send the email and password to the backend, once validation checks have been successful. The backend should look for a user matching the email and the password sent against the stored email and hash associated with the user. If the user is not found or the password does not match an error should be returned and properly shown on the frontend (refer to the screenshots below).

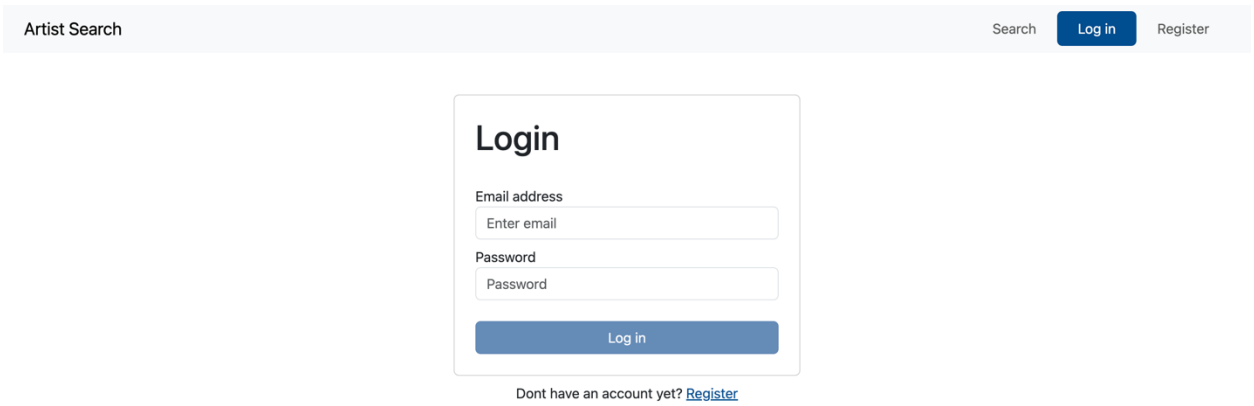
If a user is found and the password matches the hash, the user should be authenticated by responding with all required data to set up his session (maybe with user profile data). Like with the registration process, use a JWT token and HTTP-only cookies with expiration offset set at 1 hour (for both JWT and the cookie).

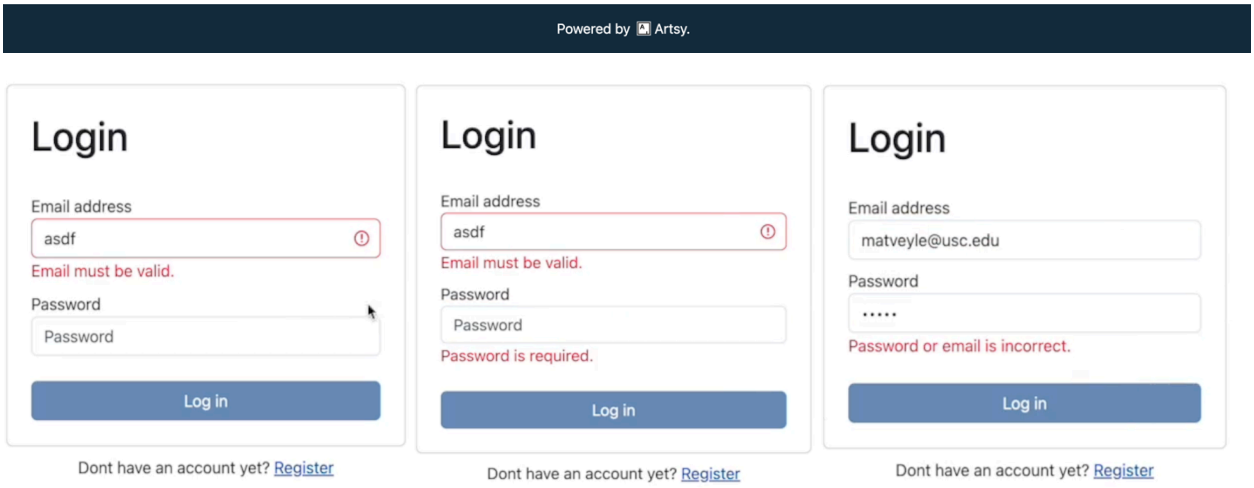
The login page (and the menu) should be available to unauthenticated users only and should match the screenshot below.

Form validation and behavior should be like the form described in the “Fixme-1. Registration” section – all fields are required, email should be valid, “Login” button should be disabled until there are no validation errors.

On a successful login, the frontend should redirect the user to the Search page and change the visual appearance of all elements according to a new auth state (described below).

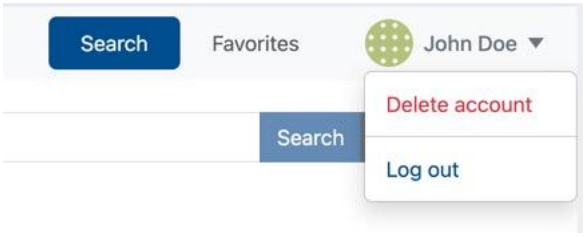
The **Register** link should lead to the registration page.




## 4-6-3 Logout Procedure

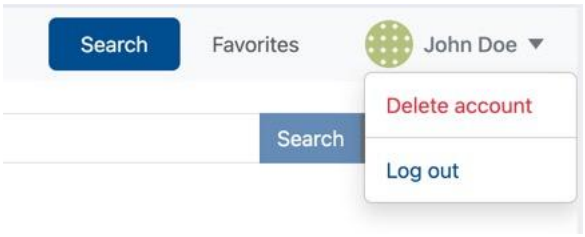
This menu should be available to authenticated users only and should match the screenshot below. When an option is clicked, it should send a request to the backend to clear the cookie. Additionally, the frontend should reflect a new unauthenticated state. If the user was previously on the page that requires authentication, redirect to the Search page.




## 4-6-4 Delete account Procedure

This menu should be available to authenticated users only and should match the screenshot below. When this option is clicked, it should send a request to the backend to delete the user with all data

associated and clear the cookie. Additionally, the frontend should reflect a new unauthenticated (guest) state. If the user was previously on the page that requires authentication, redirect them to the Search page.




### 4-6-5 Current auth state

Once the user logs in, the page refresh should not affect the frontend's auth state (until the user logs out / deletes his profile)!

Frontend in the authenticated state must store some profile data (profile, favorites), hence after the page is refreshed, the frontend should be able to restore that data. There could be different approaches:

1) "…/me" request (Recommended).
   This approach suggests the following – once the page is loaded, the frontend immediately issues a specific request to the backend. This includes an additional endpoint (usually "…/me") that tries to authenticate users based on the JWT cookie (if provided) and returns some state (could be current user profile and maybe some other data) that usually has data like the "/login" endpoint.
   If there are no credentials provided (or they have expired/ are invalid) the corresponding response indicates to the frontend that the current state is "unauthenticated".
   If user data is returned – frontend can just use this data and change auth state to "authenticated".
2) Local storage. We don't recommend persisting user data in the localStorage, since it might be impossible to determine the consistency of localStorage data against the data in the HTTP-only Cookie (ex: it can be expired or cleared), since JS cannot read it. There are some workarounds, however they could simply lower the security.

## 4-7 User Favorites

This assignment allows authenticated users to store a list of favorite artists. The backend should keep this list associated with user (and store it in the database) and support the following actions:

1) Adding an artist to the user's favorites list
2) Removing an artist from the user's favorite list
3) Retrieving user's favorites

The backend should also track the moment of time when the artist was added. Later, the frontend should display them in the "newest-first" order.

Since the list is associated with the user, it should be deleted once the user is deleted.

This list will affect the appearance of certain elements, so it might be convenient to return this data along with the user profile. In other words, the frontend should have this list (and maintain its

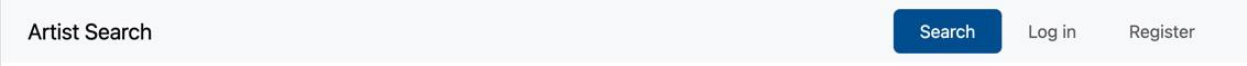
consistency) as soon as possible to correctly display artists' state on the screen. Visual description is provided in the corresponding section below.

## 4-8 Page Layout

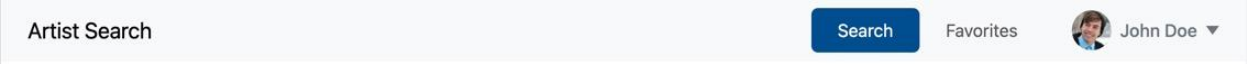
The page contains a header section, a content section and a footer section arranged vertically.

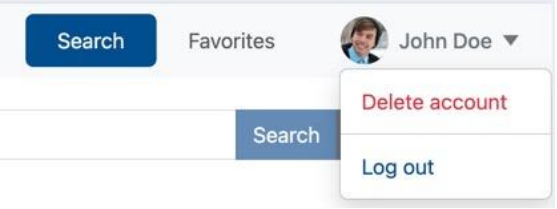
The header section has a bar (Hint: Check Bootstrap navbar component) containing the site title "Artist Search" on its left and a menu on the right. Items available in the menu depend on whether the user is logged in or not.

For unauthenticated users - Search, Log In, Register:



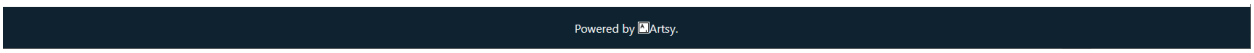

For authenticated users - Search, Favorites and a profile dropdown:




Profile dropdown consists of a static part and a dropdown itself. Static part should have user's avatar (from Gravatar.com), full name and a caret. Click on the static parts triggers dropdown and it opens/closes. Any click outside the dropdown, as well as the "ESC" key press closes the dropdown. Dropdown consists of two items: "Delete account" and "Log out". Those options are described in the "FIXME-4 Authentication" section.

The footer section is another static bar containing Artsy attribution "Powered by Artsy.", in which the Artsy logo is placed before the word "Artsy". When Artsy text on the footer is clicked, the page is redirected to Artsy homepage.




Page Footer

The content section contains the contents of the page, in which the following features will be shown.

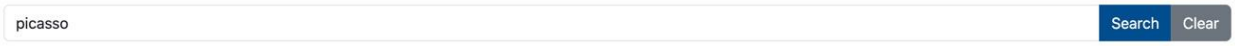
## 4-9 Search Form

When a user opens the page for the first time, there is only a search form (Hint: Check Bootstrap Forms) in the content section. Search form contains three items: an input section, a search button and a clear button, shown below.


Search Form

The behavior of the **Clear** button is described in Section 4-12.

The Search button is enabled if and only if the input section contains some input as shown below. (Hint: check **disabled** attribute).



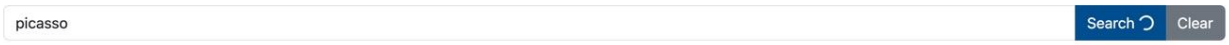

Search Button is Enabled when Input Section is not Empty

Artist names will be entered into the input section of the search form. The input section has the placeholder value “Please enter an artist name.”, which is shown when the input field is empty. When the input section is focused (i.e., input section is clicked and the cursor is blinking within the field), its style changes as shown below.


Input Section when Focused

Users can initiate a search by either **1) clicking the search button** or **2) by hitting the enter key**. Users cannot initiate search when the input section is empty because the search button is disabled.

Like Assignment 2, when the search is initiated, your frontend will do an AJAX call to the backend, sending the input text. Upon receiving the request, your backend will make a request to Artsy Search API, forwarding the input text to Artsy Search API via parameter **q** as described in the Search Endpoint section above. Until a response is received from the backend, a spinner (Hint: Check Bootstrap spinners) will be shown near the search button as shown below.



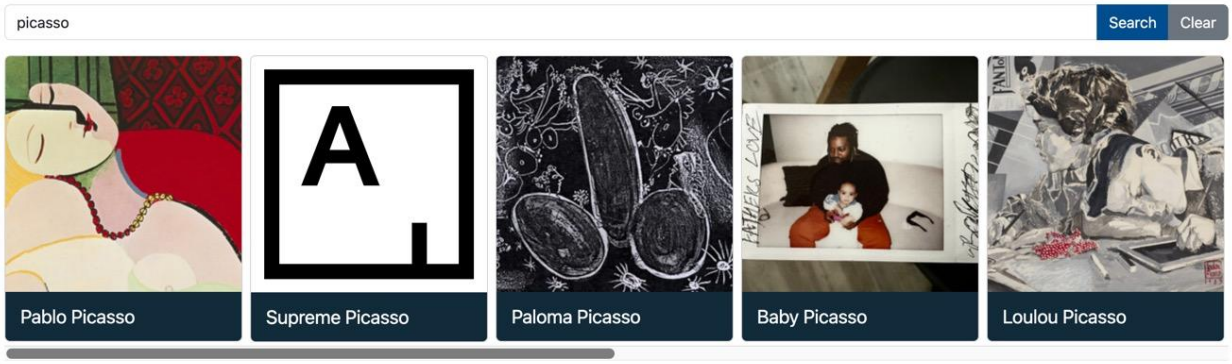

Spinner is Shown Until a Response Arrives

When the response arrives, the spinner will be hidden, and the results are shown in the result list.

The clear button of the search form brings the page to its initial state. It clears the input section, result list and artist details. Consequent page refresh shows the initial state of a cleared page.

## 4-10 Result List

The results of the search are shown as a list of cards (Hint: Check Bootstrap Cards), similar to Assignment 2. Each card contains the artist image with link retrieved from the ["_links"]["thumbnail"]["href"] field of an artist resulting in the response of search endpoint. Below the image, the artist’s name is listed corresponding to the [“title”] field of the artist result. The list of result cards is scrollable. The artist names have a blue background color (HTML Color Code: #205375).




Result List

If a particular artist does not have an image, Artsy returns “/assets/shared/missing_image.png” in the ["_links"]["thumbnail"]["href"] field as mentioned in Section 4-2-2. In this case, you will place Artsy logo as the artist image as shown below.


Artsy Logo is Shown if an Artist Does Not Have an Image

If the search results are empty, i.e. no artists match the given query, an error message containing text “No results.” will be shown as below (Hint: Check Bootstrap Alerts).


No Artists Match the Query

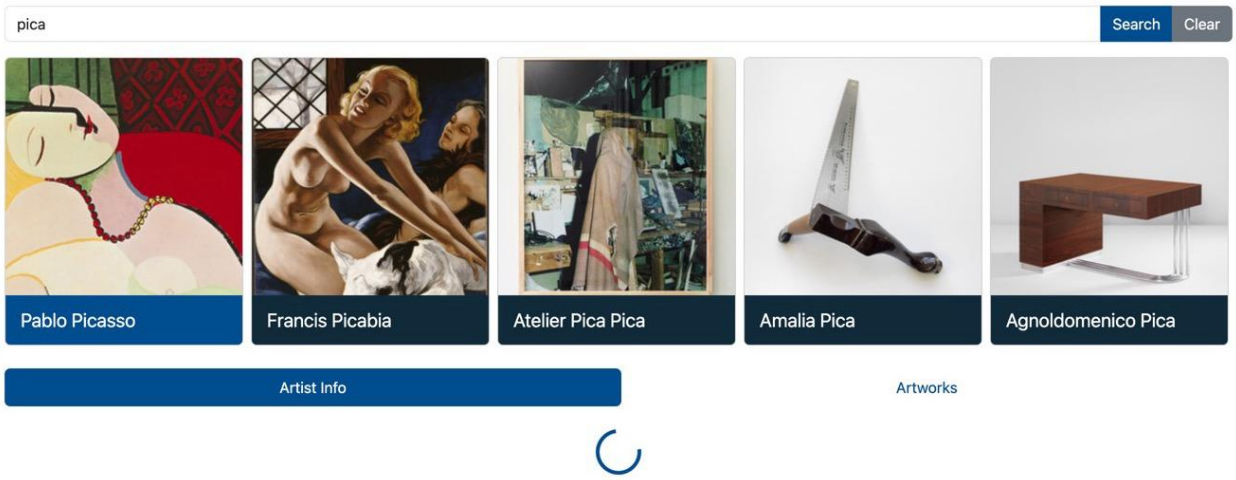
When a card is hovered, its background color changes as shown below.


The Background Color Change of a Hovered Card

When the user is authenticated, the artist card should contain a button with a star icon. If the artist is not in favorites, the star should have no fill. If the artist is in favorites – start should have a yellow fill. Click on the button toggles the state – adds and removes an artist to/from favorites. This change should be reflected immediately everywhere it is shown (for example, in the Artist Info tab) and trigger the notification logic described in the favorites section below.


The “favorites” button appearance and on-off states

When an artist card is clicked, your frontend will do an AJAX call to your backend to get artist details. To get artist details, you must send an artist ID to the backend. For that, you must associate artist IDs with cards, which you can do during initial search by sending artist IDs from your backend to the frontend as a part of the response. Detail view will contain two tabs – “Artist Info” (Selected by default) and “Artworks” Until a response is received from the backend, a spinner is shown for both tabs as shown below.




Spinner is Shown Until a Response Arrives

The selected card's background color is permanently set to lighter blue after the click, which denotes the active card for which details are being shown. It should stay the same until another card is selected or details are closed (by clicking on "search" or "clear" buttons).

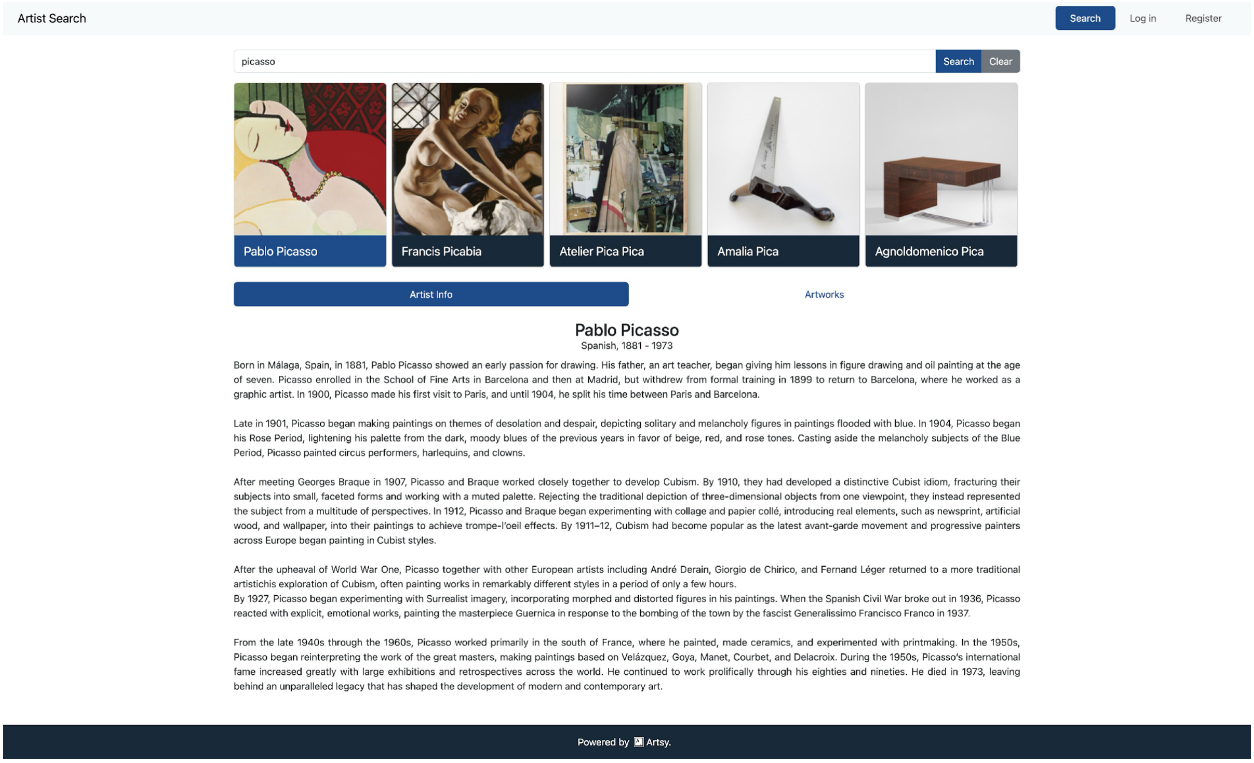
When the response with necessary data arrives, hide the spinner and show the contents of a corresponding tab.

## 4-11 Artist Details

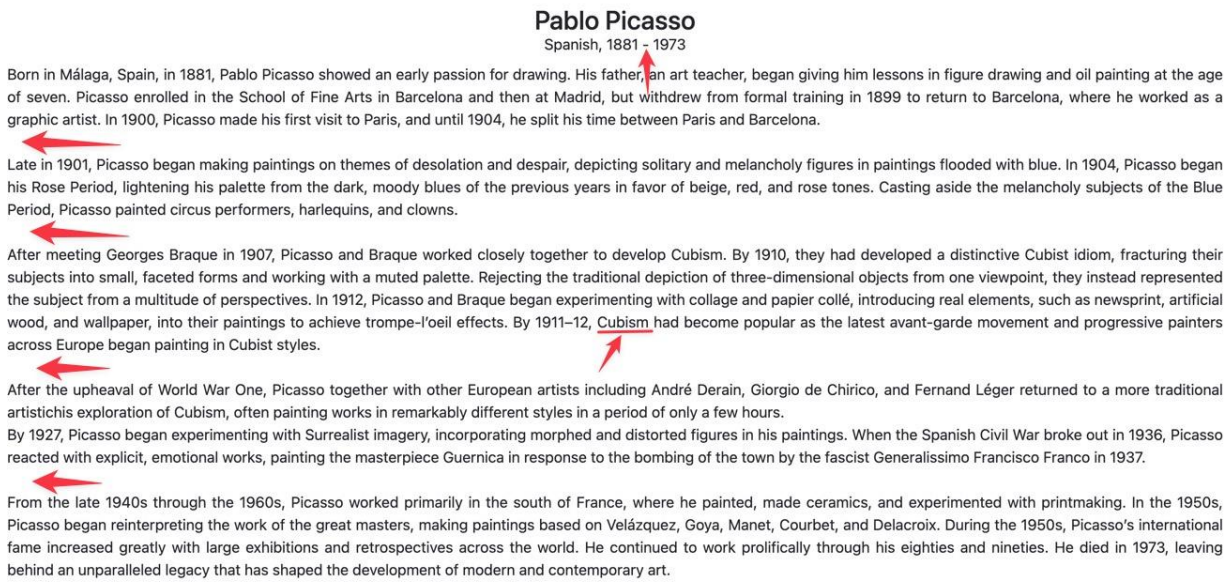
Artist details are shown in two tabs: Artist Info and Artworks (Hint: Check Bootstrap Navs and Tabs).

### 4-11-1 Artist Info Tab

Artist Info tab contains the same information as Assignment 2. It includes artist name, artist birth year, artist death year, artist nationality and artist biography retrieved from **Artists endpoint** as shown below.




Artist Info Tab




Artist Info formatting.

The first line contains the name of the artist. Next line contains the nationality of the artist, followed by birth and death days. Notice the “en dash” between dates. Next, the biography is shown. Notice proper paragraph separation and no split words for a line break (See “By 1911-12, Cubism”. Original text: “By 1911–12, Cub- ism”).

This information corresponds to [“name”], [“birthday”], [“deathday”], [“nationality”] and [“biography”] fields of the artist's endpoint response. Similar to Assignment 2, if any of the fields are empty, you should simply leave them blank.

If content does not fit the screen, the whole page should be scrollable.

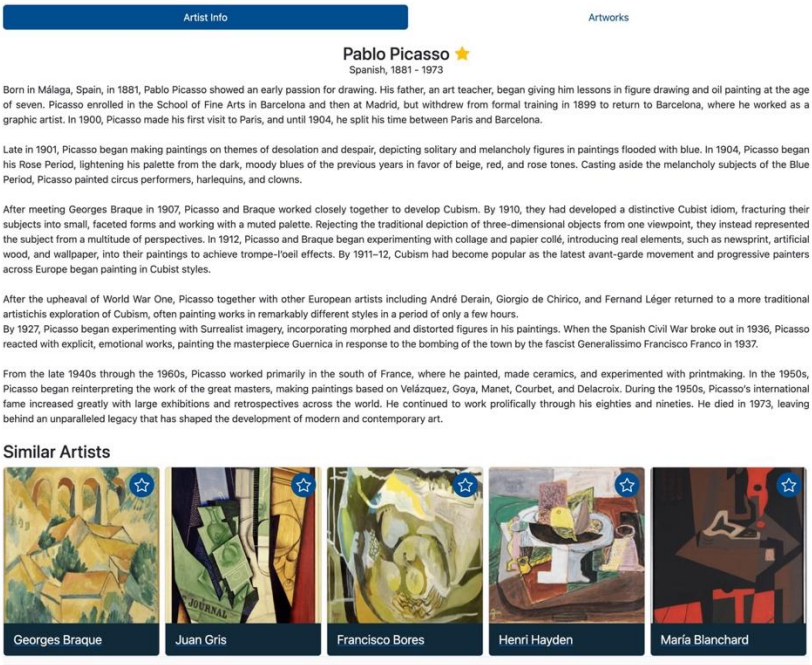
Open description (tabs and their contents) should be restored after the page refresh, however search results should hide, and search input should reset. You should design and implement it the way, one may easily share the description with others (e.g. via socials) by url only (the page url should contain enough information for frontend to restore the page state).

For authenticated users there are two main changes – a “star” button and similar artist section.

The “star” button should behave like the “star” button on an artist card.


Two states of the “star” button – not in favorites, in favorites

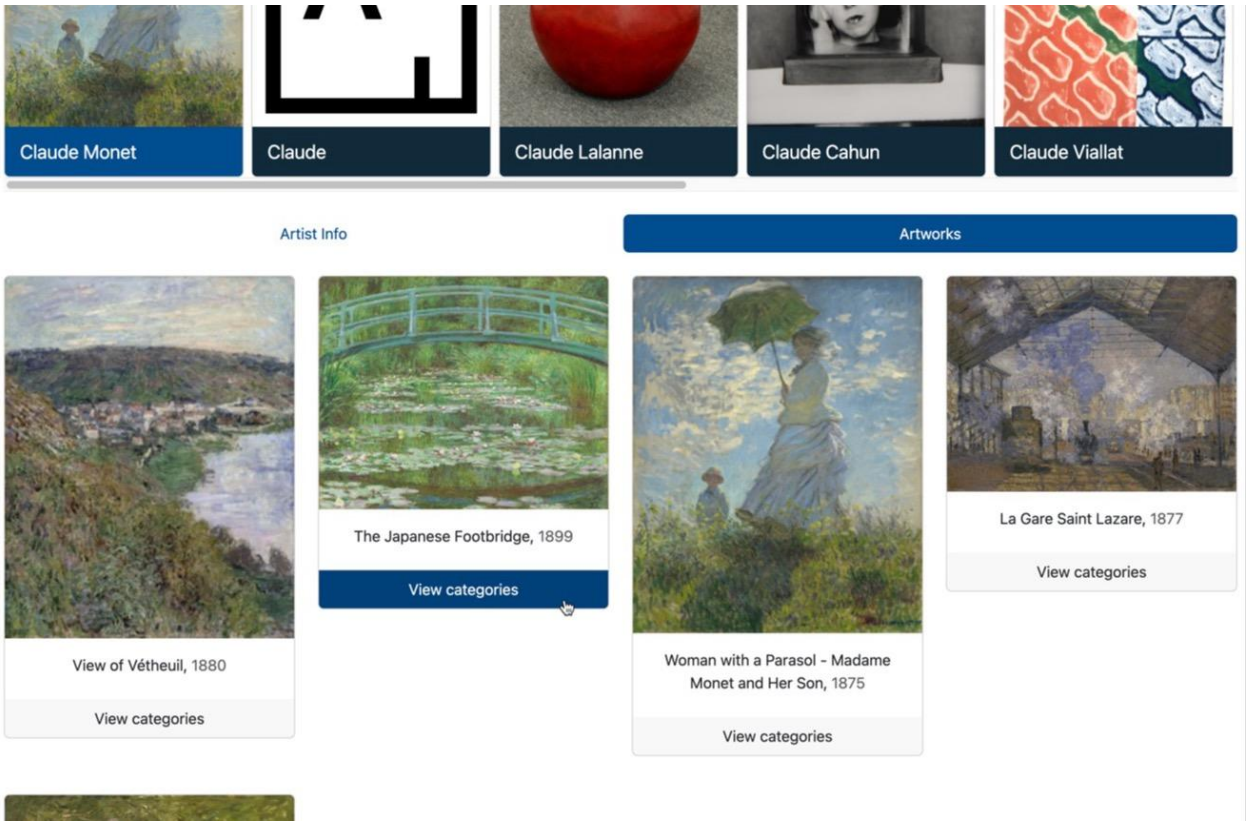
Under the description with biography there should be a block titled “Similar Artists”. Please use the corresponding artsy endpoint to get the list from your backend. Similar artists should be displayed in the form of cards that should look and behave exactly as artist cards in the search result section. You can use the same frontend components. Upon click (except on the star button), they should change the content of sections above to the artist from the card. Leave search results unchanged (but reset selected state if description does not match the artist currently being displayed).




Screenshot of the artist detail section under authenticated user

### 4-11-2 Artworks Tab

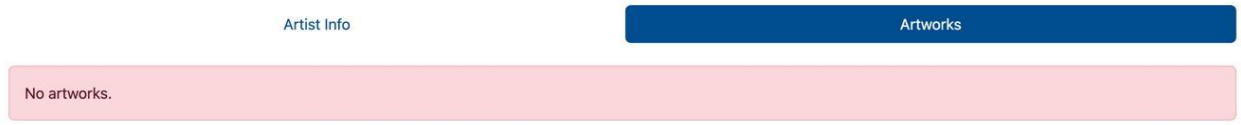
The Artworks tab contains the artworks of the selected artist. Artworks are retrieved from the Artworks endpoint described in Section 4-2-4. Example is shown below.




First Four Artworks of Claude Monet (Page Contains More)

Each artwork has an image corresponding to the [“links”][“thumbnail”][“href”] field of the artwork (See Section 4-2-4), a name corresponding to the [“title”] field of the artwork (See Section 4-2-4) and a creation year corresponding to the [“date”] field of the artwork (See Section 4-2-4). Artworks are listed using Bootstrap Cards.

If an artist does not have any Artworks in Artsy, you should show an error message containing text “No artworks.” as shown below.

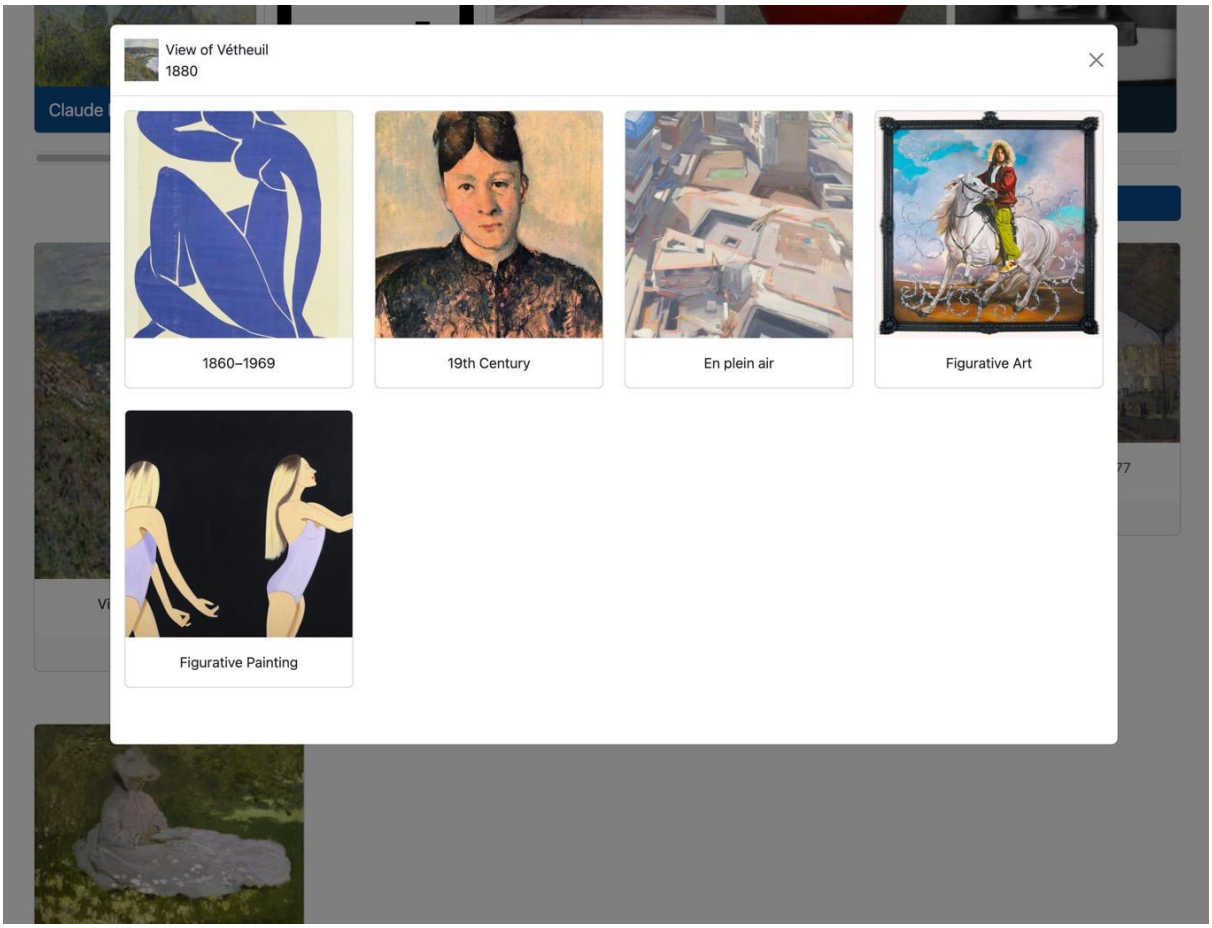



Artist does not have any Artworks

A “Categories” button below each artwork opens a modal (Hint: See Bootstrap Modals) in which categories for the artwork retrieved from the Genes Endpoint (See Section 4-2-5) are shown.

### 4-11-3 Categories Modal

The category modal has a title and a content section separated by a line as shown below.



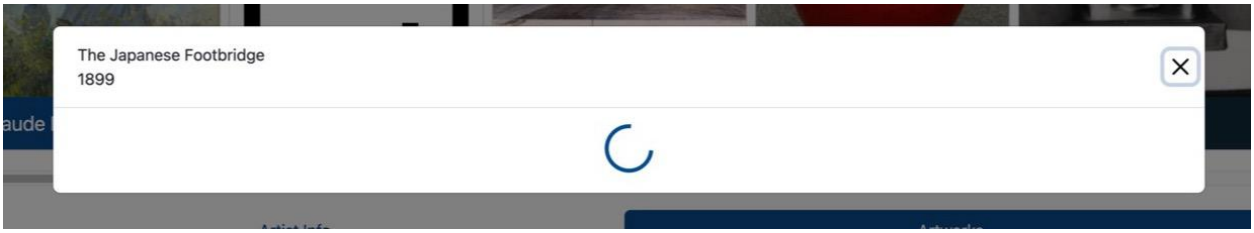

Categories Modal Sections

The title section contains the image of the artwork, name of the artwork and creation date of the artwork. The Content section contains the categories of the artwork retrieved from the Genes Endpoint.

Each category has a name corresponding to the [“name”] field of the gene (See Section 4-2-5) and an image corresponding to the [“_links”][“thumbnail”][“href”] field of the gene (See Section 4-2-5).

When the “Categories” button of an artwork is clicked, the modal is opened, and a request is sent to the backend to retrieve the categories (or genes) of an artwork. To get categories, you must send artwork ID to the backend. You can associate artwork IDs with “Categories” buttons using the IDs provided by the Artworks endpoint.

Until a response arrives, a spinner is shown in the content section of the modal as shown below. Spinner is replaced by the categories when the response arrives. Each category is shown using a Bootstrap Card.

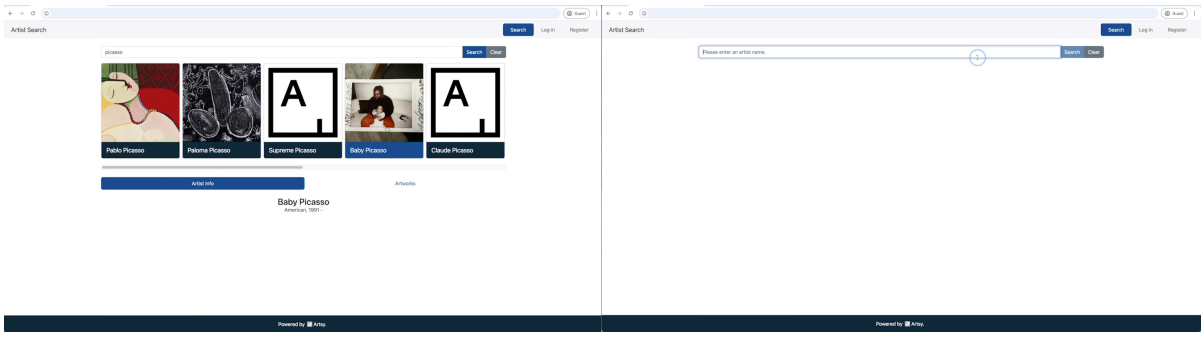


Spinner is Shown until Categories Response Arrives

If a particular artwork does not have any categories, an error message containing the text “No categories.” Is shown in the content section of the modal.

## 4-12 Clear Button of the Search Form

The clear button of the search form brings the page to its initial state. It clears the input section, result list and artist details as shown below.




Before and After clicking the ‘Clear’ button

## 4-13 Favorites page

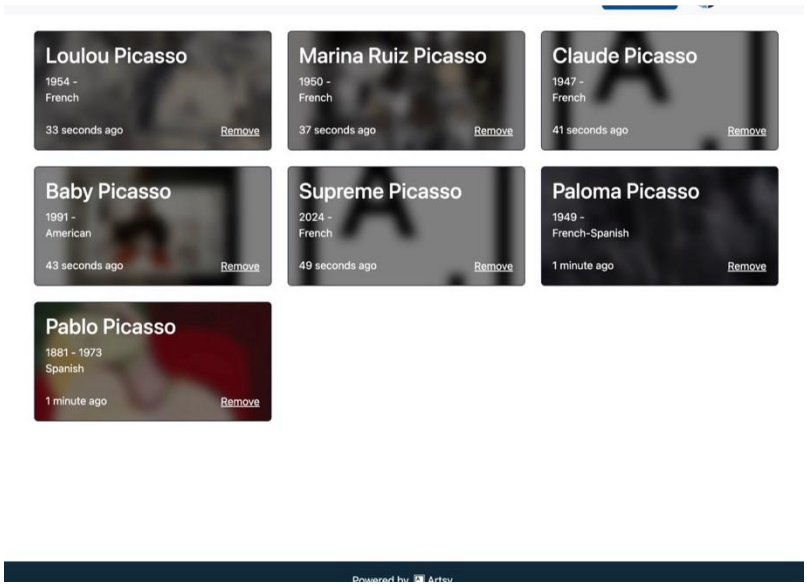
This page is only available to authenticated users.

While loading, a spinner should be shown as on the screenshot below. To improve user experience, the list of favorite artists should be maintained, so on navigation from another screen, there should be no loading from the server, hence no spinner. The loading should only occur on page (full) reload.


If there are no artist in favorites, a “No favorite artists” message should be shown.


No favorite artists message

If there are some artists in the list, they should be displayed as cards in the “newest first” order.




Favorites page cards

The Favorite artist card should contain the following information: full name, dates of birth and death, nationality, interactive timer representing relative time of when the artist was added to the favorites and a blurred background image (the same used on artist card in search results or in the similar artists list).

The **Interactive timer** is a relative time expressed with words which is auto updated. Some examples: for less than a minute, display “1 second ago”/”n seconds ago”; for more than a minute, but less than an hour display “1 minute ago” / “n minutes ago”, etc.

The Card should support two interactions:

1) On “Remove” button click artist should be removed from the favorites list
2) On card click (except on “remove” button) user should be navigated to the “Artist Details” page with the artist being displayed. Search bar should be empty and there should be no search results. Both tabs should behave identically as they behave after artist details are being shown after search and select.

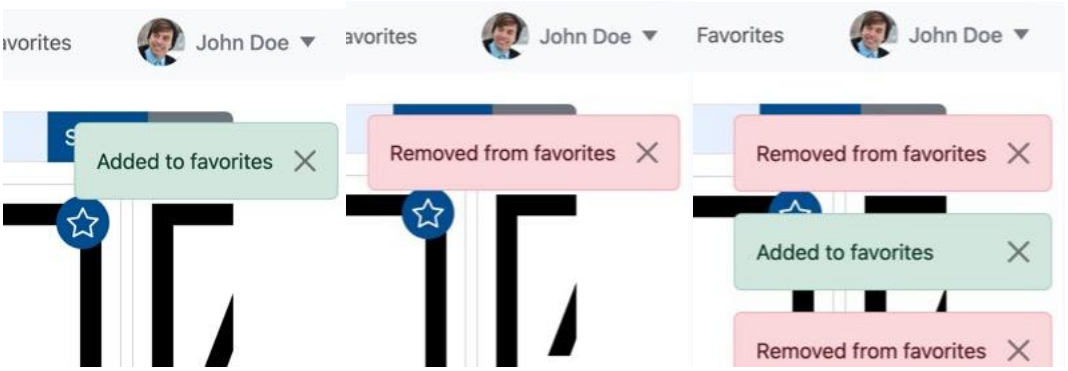
## 4-14 Notifications

Application should show confirmation in form of a top-right corner stackable notification for the following actions (with corresponding text and bootstrap style):

1) add to favorites – “Added to favorites” (style = success)
2) remove from favorites – “Removed from favorites” (style = danger)
3) logout – “Logged out” (style = success)
4) profile deletion – “Account deleted” (style = danger)

Notifications should be shown regardless of where the action has been made. These notifications should be visible for 3 seconds and automatically removed afterwards. In addition user may click “X” button on the left part of the notification to manually remove it. Notifications should stack vertically with the last

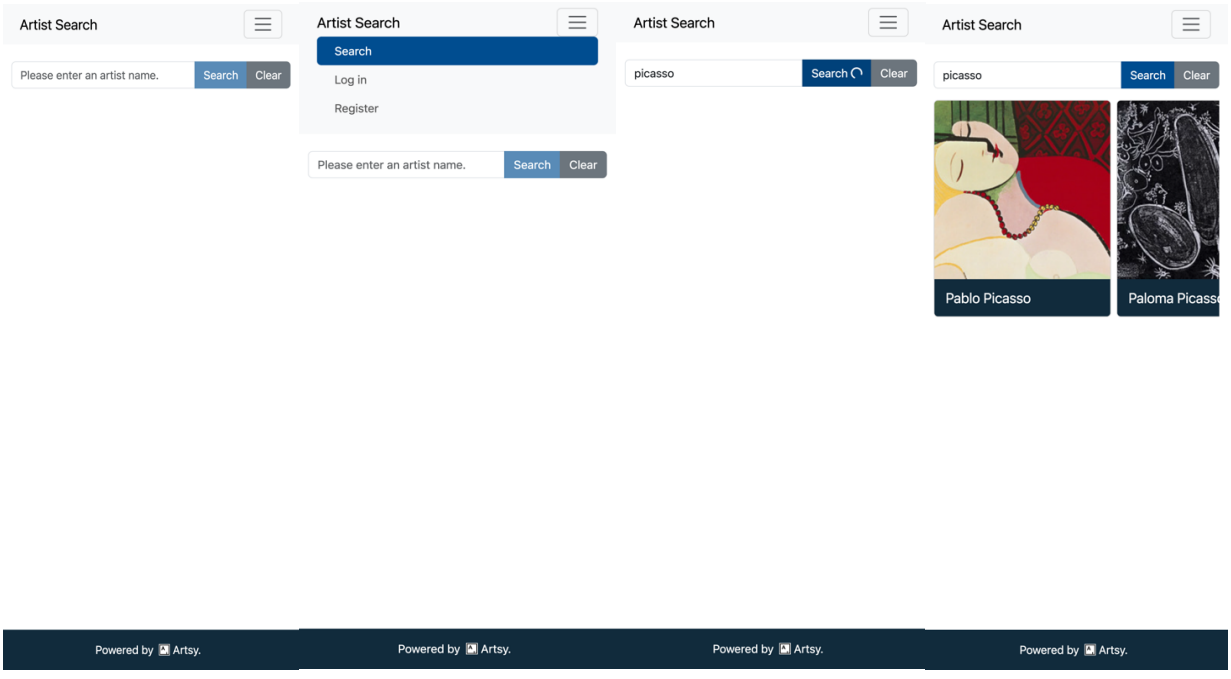
notification triggered being on the bottom of the stack and the earliest being on the top. The stack grows from the top to the bottom of the page and persists between page navigations (not page reloads).

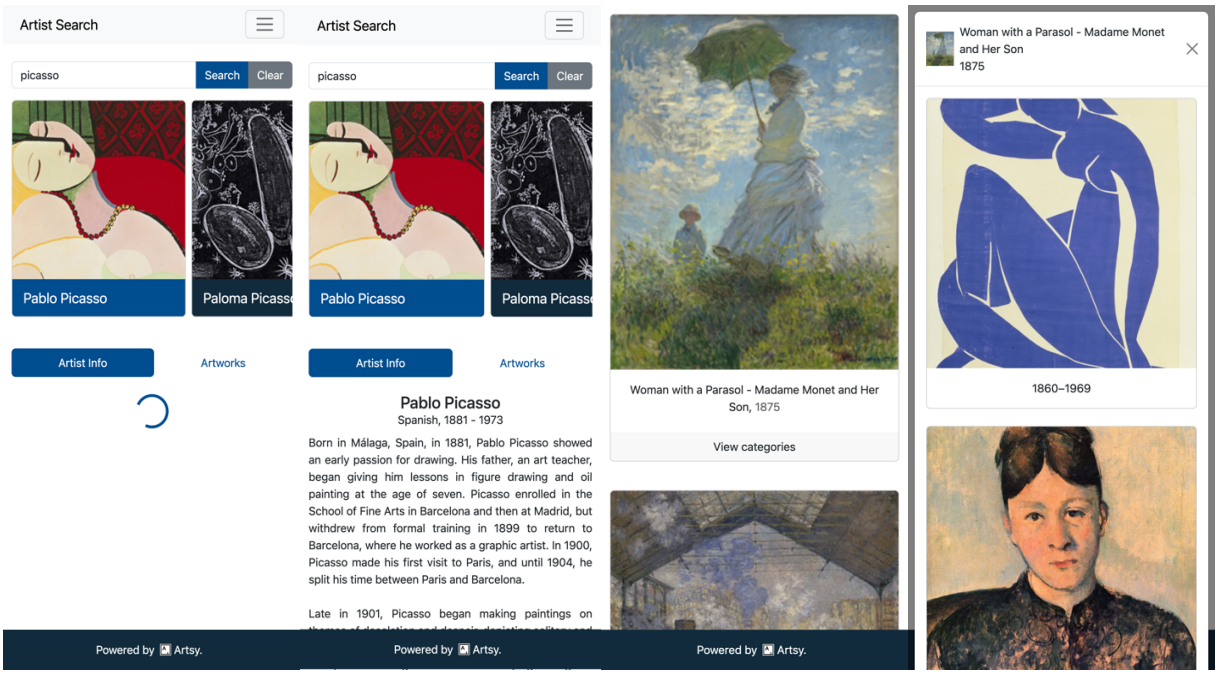


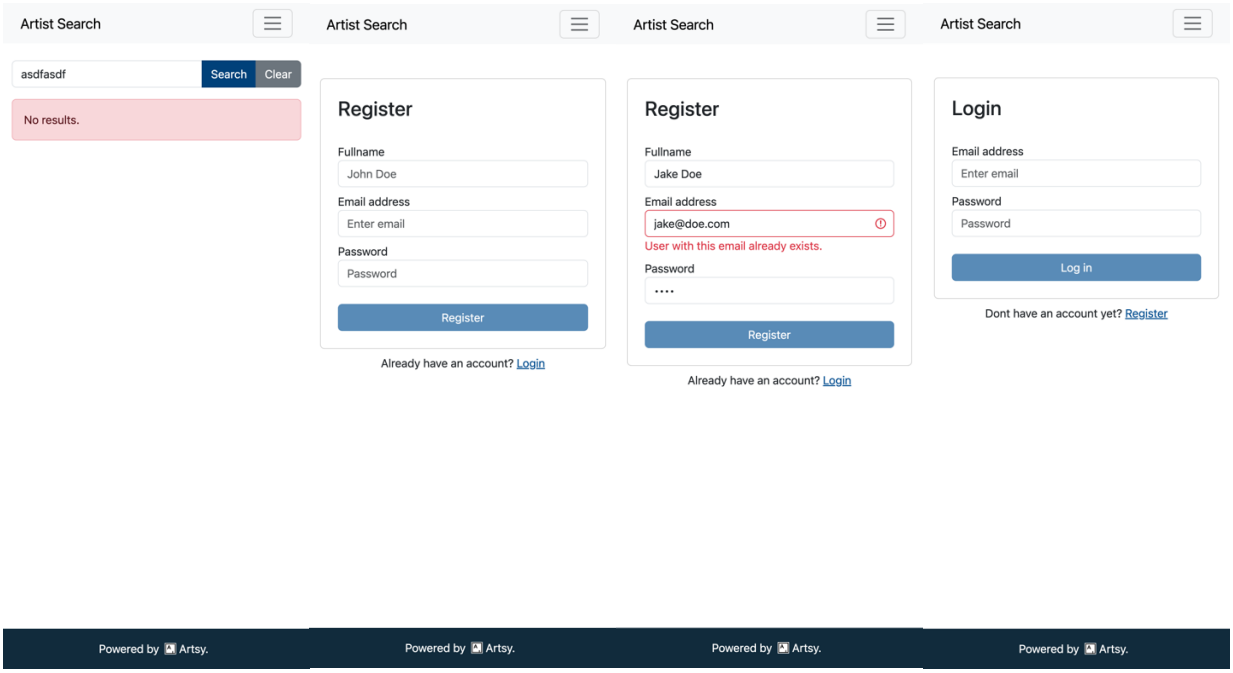

Notifications and the stack of notifications

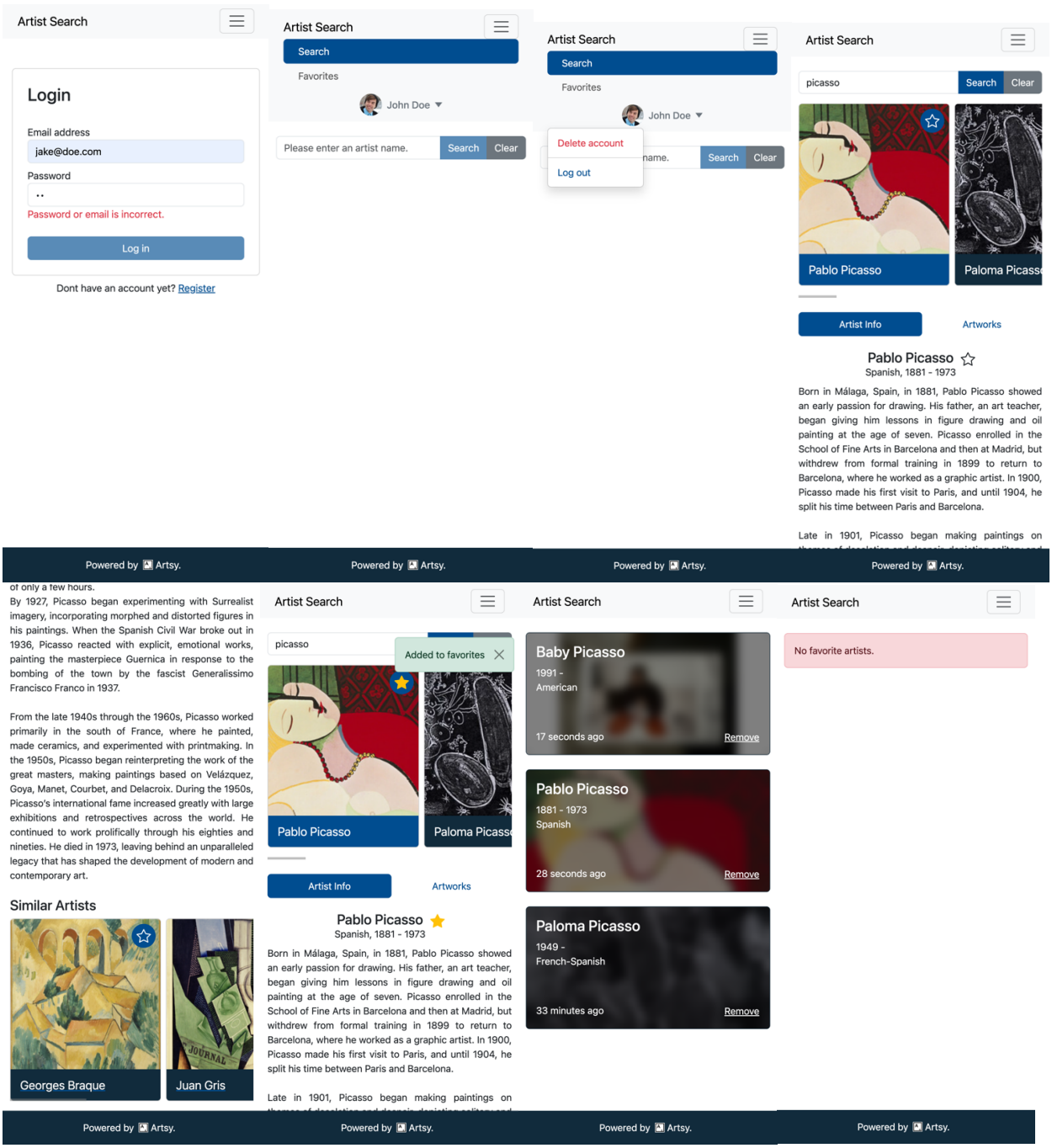
## 4-15 Responsive Design

The webpage you develop must be responsive. One easy way to test responsive behavior is to use Google Chrome Responsive Design Mode in the Developer Console. Here are some snapshots from an **iPhone 14 Pro Max** using the Google Chrome Responsive Design Mode. All functions should work on mobile devices.




Artist Search
picasso
Search
Clear
Pablo Picasso
Paloma Picasso
Artist Info
Artworks
Pablo Picasso
Spanish, 1881 - 1973
Born in Málaga, Spain, in 1881, Pablo Picasso showed an early passion for drawing. His father, an art teacher, began giving him lessons in figure drawing and oil painting at the age of seven. Picasso enrolled in the School of Fine Arts in Barcelona and then at Madrid, but withdrew from formal training in 1899 to return to Barcelona, where he worked as a graphic artist. In 1900, Picasso made his first visit to Paris, and until 1904, he split his time between Paris and Barcelona.
Late in 1901, Picasso began making paintings on
Woman with a Parasol - Madame Monet and Her Son, 1875
View categories
Woman with a Parasol - Madame Monet and Her Son
1875
1860–1969
Powered by Artsy.


Artist Search
asdfasdf
Search
Clear
No results.
Register
Fullname
John Doe
Email address
Enter email
Password
Password
Register
Already have an account? Login
Jake Doe
jake@doe.com
User with this email already exists.
Login
Log in
Dont have an account yet? Register
Powered by Artsy.




## 5 Assignment Submission

In your Table of Assignments page (on GitHub Pages), update the Assignment 3 link to refer to your deployed website of this assignment. Also, similar to Assignment 2, provide an additional link to one of your cloud query entry points, below the homepage link. Your project must be hosted on Google Cloud. Graders will verify that these links are indeed pointing to the cloud service.

Also, submit your source code file to D2L Brightspace as a **ZIP archive**. Include your frontend and backend source code, plus any additional files needed to build your app (e.g., yaml file). The timestamp of your last submission will be used to verify if you used any grace days.

## 6 Notes

- You cannot use jQuery for Assignment 3.
- You must use Bootstrap for Assignment 3. **If Bootstrap is not used, a 4-point deduction will be applied.**
- Appearance of the webpage should be like the reference videos as much as possible.
- Artsy lacks information for a lot of artists. Here are some examples for testing:
    - Pablo Picasso: Has every detail in Artist Info, has a single artwork.
    - Vincent van Gogh: Has everything except biography in Artist Info, has 10 artworks.
    - Yayoi Kusama: Has everything except death date (Since she is not dead) in Artist Info, has no artworks.
    - Claude Monet: Has every detail in Artist Info, has 10 artworks.
    - Raphael: Has everything except biography in Artist Info, has 10 artworks, some of which do not have categories.

## 7 Hints

- To allow your frontend to call the backend without triggering CORS issues, it's usually easier to serve API endpoints under the same hostname with the `/api/...` prefix. This way, you don't need to modify your frontend configuration before deploying your app to GCP. In this setup, your frontend can call `/api/...` directly without specifying a fully qualified URL (e.g., `fetch("/api/")`), as the frontend's origin will automatically be used.
- For local development and debugging, you can enable proxying all `/api` calls to your backend using the `proxy.conf.json` configuration file. Vite (which is part of the Angular toolkit) will use this file to set up a proxy. Please refer to the documentation here: https://v17.angular.io/guide/build#proxying-to-a-backend-server (For angular 17). Ensure your backend is running before making requests.
- Formatting relative time is a complex task since different languages and countries have varying rules and grammar. Browser JavaScript engines include a built-in internationalization module called Intl. You can use the new `Intl.RelativeTimeFormat('en', …)` function to correctly format time differences. Since your formatter should return different time units, you may want to use a series of if statements to determine whether the time difference should be displayed in seconds, minutes, etc.
- For global notifications, consider creating a separate service that stores current notifications and removes them after 3 seconds. Then, create a component to render the list of notifications and mount it in the top-level component (e.g., app.component.ts), ensuring it remains unaffected by route changes. Avoid placing your notification component inside pages that are conditionally displayed based on router outlet - mount it in a global layout instead.
- You can create an `isLoggedIn` RxJS Subject in the AuthService to allow different parts of your app to react to authentication status changes. This can be useful for showing/hiding UI elements and reloading the favorites list upon login.

- Consider using two backend middleware functions: authenticated and unauthenticated to protect specific endpoints.
    - The authenticated middleware should check for the presence of an authentication cookie, validate the JWT, store the user identifier in the request context, and throw an HTTP error if authentication fails. Use this middleware for protected routes like logout, delete profile, and favorites.
    - The unauthenticated middleware should block authenticated users from accessing certain routes, such as login and register, by throwing an HTTP error if an auth cookie is detected.

## 8 Static Resources

- You can find the Artsy logo required for this Assignment in D2L Brightspace.
- You may use any star and star-filled icons in svg format. For example:
  https://fontawesome.com/icons/star?f=classic&s=regular
  https://fontawesome.com/icons/star?f=classic&s=solid